

Informe técnico — Primera entrega TPI Food Store

Resumen de lo actuado en Unidades 1, 2 y 3 · verificación en PostgreSQL 18 · IA: OpenCode (+Kiro no disponible)

1. Objetivos del TPI y evidencia entregada

#	Objetivo	Evidencia	Verificación
1	Modelo ER (entes, atributos, claves, cardinalidad, participación)	01_mer_relacional.md	Revisado vs. enunciado R1-R7
2	Paso de ER a relacional (1:N por FK, N:M por tabla intermedia)	01_mer_relacional.md	Consistente con ddl.sql
3	Normalización hasta 3FN/BCNF con dependencias funcionales	02_normalizacion.md	DF (pedido_id, producto) → precio_unitario
4	DDL: tipos, PK/FK, restricciones, índices	ddl.sql (= schema.sql)	Ejecutado en base limpia, 0 errores
5	DML y consultas: JOIN, agregación, subconsultas, HAVING, ventanas	consultas.sql + 03_dml_consultas.md	Q1-Q10 ejecutadas en food_store_tp3
6	Vistas, funciones y procedimientos PL/pgSQL	objetos.sql	3 vistas + 3 funciones + CALL sp_registrar_pedido
7	Reglas de negocio con CHECK, UNIQUE y triggers	triggers_restricciones.sql	3 triggers probados en motor
8	Transacciones: atomicidad, aislamientos, concurrencia	04_transacciones.md + transacciones_prueba.sql	3 aislamientos + FOR UPDATE + deadlock
9	Borrado lógico y su impacto en consultas e índices	05_borrado_logico.md + borrado_logico_prueba.sql	50.000→49.999, histórico intacto

2. Unidad 1 — Modelo de datos e integridad

Qué se hizo: se derivó el modelo ER del enunciado del negocio (5 entidades: categoria, cliente, producto, pedido, detalle_pedido) aplicando las reglas R1–R7. La relación N:M pedido–producto se resolvió con la entidad asociativa detalle_pedido, que además guarda cantidad, precio_unitario (precio capturado al momento de la venta, R4) y subtotal.

Normalización: la planilla de ventas se normalizó hasta BCNF. La dependencia funcional clave de la planilla es (pedido_id, nombre_producto) → (cantidad, precio_unitario, subtotal); se separaron fecha/cliente/forma_pago (dependen solo de pedido_id) y categoria (depende del producto). Resultado: todas las tablas del esquema en BCNF.

DDL: ENUM forma_pago_enum (EFFECTIVO/TARJETA/TRANSFERENCIA), IDENTITY para claves sustitutas, FK con ON DELETE RESTRICT, CHECK (precio y stock ≥ 0), UNIQUE (email, nombre de categoría), TIMESTAMPTZ y NUMERIC(14,2) para montos.

```
food_store_tp3=> \i Parcial/ddl.sql

CREATE TYPE
CREATE TABLE categoria
CREATE TABLE cliente
CREATE TABLE producto
CREATE TABLE pedido
CREATE TABLE detalle_pedido
CREATE INDEX ... (5 tablas, 6 índices)
-- 0 errores: esquema creado de punta a punta sobre PostgreSQL 18
```

3. Unidad 1/2 — Integridad, transacciones y concurrencia

Integridad (triggers/funciones plpgsql): baja lógica anti-DELETE físico sobre categoria y producto; descuento de stock con validación de disponibilidad en detalle_pedido; verificación de subtotal = cantidad × precio_unitario.

Concurrencia (2 sesiones psql): se reprodujo una lectura no repetible (READ COMMITTED vs REPEATABLE READ), lectura fantasma (READ COMMITTED vs SERIALIZABLE), bloqueo FOR UPDATE y un deadlock, todo cerrado con ROLLBACK.

```
food_store_tp3=> (Sesion A)

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT stock FROM producto WHERE nombre='Producto 1';
  stock
-----
    10

-- Sesion B: UPDATE producto SET stock=stock+500 ... COMMIT;

SELECT stock FROM producto WHERE nombre='Producto 1';
  stock
-----
    50 <-- lectura no repetible (A ve el COMMIT de B en el medio)
ROLLBACK;
```

Resultado medido: la sesión B de FOR UPDATE quedó bloqueada durante ~3,4 s hasta que A hizo ROLLBACK. Atomicidad verificada: un DELETE físico aborta la transacción completa por el RAISE EXCEPTION del trigger.

4. Unidad 2 — Optimización de consultas

Qué se hizo: consultas de reporting reales (4 tablas) sobre la base masiva, medición con EXPLAIN ANALYZE y reescritura con agregación anticipada + índice de cobertura. Se descartaron las propuestas que no mejoraron el plan.

Consulta	Antes	Después	Ganancia
C1 facturación por categoría/mes (TP4 P1)	4387 ms	2052 ms	2,1×
C2 ranking de clientes por gasto (TP4 P1)	754 ms	751 ms	sin mejora — índice descartado
Resumen de ventas por producto (TP3 P2)	994 ms	290 ms	3,4×
Competencia por compra (TP3 P5)	39,9 ms	5,7 ms	7,0×

Equivalencias verificadas con EXCEPT = 0 filas: RANK() vs. subconsulta correlacionada replicando RANK; subconsulta correlacionada vs. JOIN+GROUP BY; versiones con JOIN explícito vs. implícito en TP3 P4. Ajuste de la consulta 6 en la revisión final: el ORDER BY sobre la columna calculada colgaba (costo ~22M); con el outer acotado a top 5 por precio ejecuta en ~53 ms.

5. Unidad 3 — Índices, vistas y objetos programables

Índices (medición antes/después con EXPLAIN ANALYZE): 3 índices justificados en specs más propuestas de IA descartadas por redundancia (idx_producto_categoria) y sin cambio de plan (pedido(cliente_id, fecha)).

Objeto	Antes	Después	Ganancia
Búsqueda catálogo por nombre (text_pattern_ops, parcial WHERE activo)	Seq Scan 20 ms	Bitmap 8,4 ms	Seq→Bitmap
Cliente por apellido (índice btree)	Seq 15,9 ms	Index 0,9 ms	~18×
Pedidos tarjeta por mes (forma_pago, fecha)	78,4 ms	6,8 ms	sin remoción del 33%
Vista materializada facturación c/mes	2447 ms	~15 ms	~163×

Costo de escritura aceptado: 300 INSERT pasaron de 47,5 ms a 151,6 ms (+3,2×) por el mantenimiento de índices. La MV se refresca con REFRESH CONCURRENTLY (corte mensual) y queda congelada entre refrescos.

Procedimiento PL/pgSQL invocado con CALL (requisito PostgreSQL 16+): sp_registrar_pedido recibe arrays de productos y cantidades, inserta pedido + detalle en un loop y deja que el trigger descuente stock. Probado dentro de BEGIN...ROLLBACK.

```
food_store_tp3=> BEGIN; CALL sp_registrar_pedido(1, 'TARJETA', ARRAY[1,2], ARRAY[1,1]);

CALL
NOTICE: Pedido 200602 registrado para el cliente 1
-- dentro de la transaccion crecieron pedido y detalle_pedido
-- y el trigger descontó stock del producto 1
ROLLBACK;
```

Vistas: catálogo vigente con nombre de categoría, pedidos con datos del cliente (sin teléfono para no exponer dato sensible) y detalle facturado con nombre del producto.

6. Unidad 3 — Borrado lógico (soft delete)

R7: categoria y producto usan columna activo; un trigger BEFORE DELETE impide el borrado físico. La baja se hace con UPDATE activo=FALSE y las consultas filtran por ello (vista vigente e índice parcial WHERE activo).

```
food_store_tp3=> (demo borrado lógico)

--- 1. Estado inicial
SELECT count(*) FROM producto; -> 50000

--- 2. DELETE físico (debe FALLAR)
BEGIN; DELETE FROM producto WHERE id=1;
ERROR: No esta permitido eliminar registros fisicamente. Use UPDATE ... SET activo = FALSE.
ROLLBACK;

--- 3. Baja lógica correcta
UPDATE producto SET activo=FALSE WHERE id=1; -> UPDATE 1

--- 4. Catálogo vigente
SELECT count(*) FROM vista_productos_vigentes; -> 49999

--- 5. Historial de ventas intacto
SELECT count(*) FROM detalle_pedido dp WHERE dp.producto_id=1; -> 12

--- 6. Restauración
UPDATE producto SET activo=TRUE WHERE id=1;
```

Impacto: el histórico de ventas del producto se preserva íntegro (12 líneas de detalle) tras la baja; la vista vigente administra el catálogo activo y el índice parcial WHERE activo mantiene el índice liviano.

7. Requisitos técnicos del motor (PostgreSQL 16+)

Tablas de transición en triggers (REFERENCING OLD TABLE / NEW TABLE, FOR EACH STATEMENT): el trigger `trg_auditar_ajuste_precios` audita un ajuste masivo de precios. En la demo se actualizó toda la categoría 1 (12.500 filas) y el trigger statement hizo **un solo disparo**: 1 fila en `auditoria_precios` (total 34.453.393 → 37.898.738, diff promedio +275,63). Probado dentro de `BEGIN...ROLLBACK`.

```
food_store_tp3=> (demo tablas de transicion)

BEGIN;
UPDATE producto SET precio = precio * 1.10 WHERE categoria_id = 1; -> UPDATE 12500

SELECT * FROM auditoria_precios;
 id | categoria_id | n_productos | total_antes | total_despues | diff_promedio
-----+-----+-----+-----+-----+-----
  1 |           1 |       12500 | 34453393.22 | 37898738.58   |        275.63
(1 fila) <-- 1 disparo por STATEMENT (no por fila)
ROLLBACK;
```

JSONB (datos semiestructurados de producto): tabla auxiliar `producto_extra` con origen, vegano, etiquetas y info nutricional. Se demostraron los operadores `->>`, `->`, `@>` (filtrar por etiqueta) y el acceso a valores anidados con cast booleano.

```
food_store_tp3=> (demo JSONB)

SELECT p.id, p.nombre, e.info->>'origen' AS origen,
       (e.info->'info_porcion'->>'kcal') AS kcal
FROM producto_extra e JOIN producto p ON p.id=e.producto_id;
 id | nombre      | origen    | kcal
-----+-----+-----+-----
  1 | Producto 1 | local    | 850
  2 | Producto 2 | importado | 420
  3 | Producto 3 | local    | 120

-- filtro con @> (etiqueta 'sin azucar') -> Producto 3
-- filtro vegano -> Producto 2 y Producto 3
```